

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)

Assessment Tool Weightage: 35%

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. **Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.
2. **Design and implement an Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.
3. **Develop a Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.
4. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.
5. **Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.
6. **Design and develop a Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

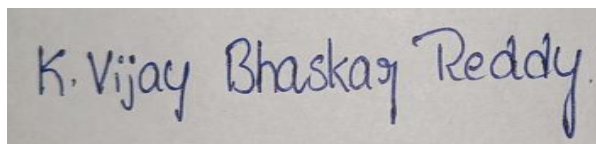
7. **Develop a Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.
 8. **Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.
 9. **Develop a Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.
 10. **Design, implement, and evaluate a Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.
-

Code of Conduct:

I K.Vijay Bhaskar Reddy(192425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in blue ink on a light-colored surface. The signature reads "K. Vijay Bhaskar Reddy." in a cursive, slightly slanted script.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1: Intelligent Traffic Signal Control (REINFORCE vs A2C)

Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

Objective:

Design a policy-gradient RL system that controls traffic signals to reduce vehicle waiting time and improve traffic throughput.

Environment:

- Four traffic queues are considered.
- The state contains normalized queue lengths and the current traffic-light phase.
- Actions:
 - 0 → Keep current phase
 - 1 → Change phase

Reward:

- Positive reward is given when vehicles are cleared.
- A penalty is given for large remaining queues.

REINFORCE:

- A Monte-Carlo policy-gradient algorithm.
- Uses complete episode returns to update the policy.
- Simple but has high variance.

A2C:

- Actor selects actions.
- Critic estimates the value of the current state.
- Uses advantage:

Evaluation:

- Average waiting time

- Throughput
- Reward convergence

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Categorical

class TrafficEnv:
    def __init__(self): self.reset()
    def reset(self): self.q = np.random.randint(0, 5, 4).astype(float); self.phase = 0; self.step_c = 0; return self._get_s()
    def _get_s(self): return np.array([*self.q/20.0, self.phase], dtype=np.float32)
    def step(self, a):
        if a == 1: self.phase = 1 - self.phase
        self.q = np.clip(self.q + np.random.poisson(0.4, 4), 0, 20)
        c1, c2 = (0,1) if self.phase==0 else (2,3)
        clr = min(self.q[c1], 3) + min(self.q[c2], 3)
        self.q[c1] -= min(self.q[c1], 3); self.q[c2] -= min(self.q[c2], 3)
        self.step_c += 1
        return self._get_s(), float(clr*0.5 - np.sum(self.q)/80.0), self.step_c >= 100

class Net(nn.Module):
    def __init__(self, is_ac=False):
        super().__init__()
        self.is_ac = is_ac
        self.fc = nn.Sequential(nn.Linear(5, 32), nn.ReLU())
        self.pi = nn.Linear(32, 2)
        if is_ac: self.v = nn.Linear(32, 1)
    def forward(self, x):
        h = self.fc(x)
        return (torch.softmax(self.pi(h), -1), self.v(h)) if self.is_ac else torch.softmax(self.pi(h), -1)

def train(algo="REINFORCE"):
    env = TrafficEnv(); net = Net(is_ac=(algo=="A2C")); opt = optim.Adam(net.parameters(), lr=1e-2)
    rewards, waits, tpts = [], [], []
    for ep in range(150):
        s = env.reset(); lps, vals, r_list = [], [], []
        done = False; total_w = 0; total_t = 0
        while not done:
            st = torch.FloatTensor(s)
            out = net(st)
            probs, val = out if algo=="A2C" else (out, None)
            m = Categorical(probs); a = m.sample()
            s, r, done = env.step(a.item())
            lps.append(m.log_prob(a)); r_list.append(r)
            if algo=="A2C": vals.append(val)
            total_w += np.sum(env.q); total_t += (r>0)
        G, returns = 0, []
```

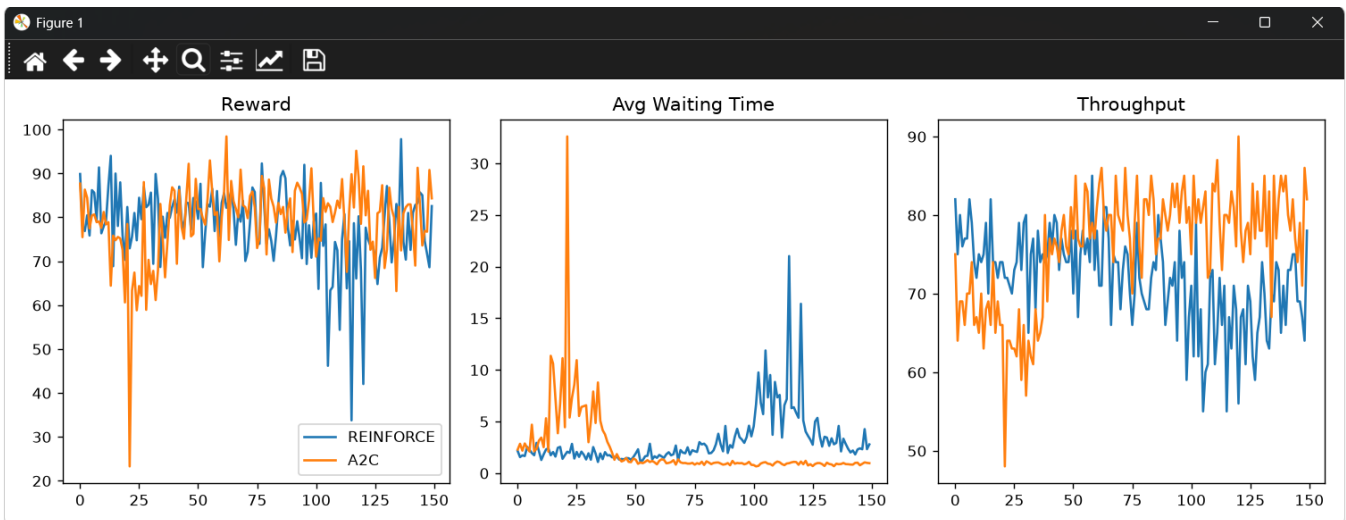
```

for r in reversed(r_list): G = r + 0.99*G; returns.insert(0, G)
returns = torch.FloatTensor(returns)
if algo=="REINFORCE":
    loss = torch.stack([-lp * g for lp, g in zip(lps, returns)]).sum()
else:
    vals = torch.cat(vals).squeeze()
    loss = (-(torch.stack(lps) * (returns - vals.detach()))).mean() + nn.functional.mse_loss(vals, returns)
opt.zero_grad(); loss.backward(); opt.step()
rewards.append(sum(r_list)); waits.append(total_w/100); tpts.append(total_t)
return rewards, waits, tpts

r_rf, w_rf, t_rf = train("REINFORCE")
r_a2c, w_a2c, t_a2c = train("A2C")
plt.figure(figsize=(12, 4))
plt.subplot(131); plt.plot(r_rf, label="REINFORCE"); plt.plot(r_a2c, label="A2C"); plt.title("Reward"); plt.legend()
plt.subplot(132); plt.plot(w_rf, label="REINFORCE"); plt.plot(w_a2c, label="A2C"); plt.title("Avg Waiting Time")
plt.subplot(133); plt.plot(t_rf, label="REINFORCE"); plt.plot(t_a2c, label="A2C"); plt.title("Throughput")
plt.tight_layout(); plt.show()
print(f"REINFORCE Final Wait: {np.mean(w_rf[-20:]):.2f} | A2C Final Wait: {np.mean(w_a2c[-20:]):.2f}")

```

REINFORCE Final Wait: 2.80 | A2C Final Wait: 0.88



Q2: Autonomous Warehouse Robot (VPG vs A2C vs A3C)

Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

Objective:

Train a warehouse robot to collect packages and deliver them to a specified location.

State:

- Robot row position
- Robot column position
- Number of remaining packages
- Whether the robot is carrying a package

Actions:

1. Move up
2. Move down
3. Move left
4. Move right
5. Pick up / deliver

Reward:

- Successful package delivery → positive reward
- Each normal movement → small negative reward

VPG:

Vanilla Policy Gradient directly updates the policy using episode returns.

A2C:

Uses an actor to select actions and a critic to estimate state value.

A3C:

Actor-Critic approach that normally uses multiple workers/environments to learn asynchronously.

Evaluation:

- Cumulative reward
- Task completion time
- Training stability

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Categorical

class WarehouseEnv:
    def __init__(self): self.reset()
    def reset(self): self.pos = [5,5]; self.pkg = 4; self.has_pkg = False; self.t = 0; return self._s()
    def _s(self): return np.array([self.pos[0]/5.0, self.pos[1]/5.0, self.pkg/4.0, float(self.has_pkg)], dtype=np.float32)
    def step(self, a):
        r, c = self.pos
        if a==0 and r>0: r-=1
        elif a==1 and r<5: r+=1
        elif a==2 and c>0: c-=1
        elif a==3 and c<5: c+=1
        elif a==4:
            if not self.has_pkg and [r,c] in [[1,1],[1,4],[4,1]]: self.has_pkg = True
            elif self.has_pkg and [r,c] == [0,0]: self.has_pkg = False; self.pkg -= 1
        self.pos = [r,c]; self.t += 1
        r_val = 20.0 if (a==4 and not self.has_pkg and [r,c]==[0,0]) else -0.1
        return self._s(), r_val, self.t >= 100 or self.pkg == 0

class ACNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(4, 64), nn.ReLU())
        self.pi = nn.Linear(64, 5); self.v = nn.Linear(64, 1)
    def forward(self, x):
        h = self.fc(x); return torch.softmax(self.pi(h), -1), self.v(h)

def train(algo="VPG"):
    env = WarehouseEnv(); net = ACNet(); opt = optim.Adam(net.parameters(), lr=1e-3)
    rewards, steps_list = [], []
    for ep in range(150):
        s = env.reset(); lps, vals, rews = [], [], []
        done = False
        while not done:
            p, v = net(torch.FloatTensor(s))
            d = Categorical(p); a = d.sample()
            s, r, done = env.step(a.item())
            lps.append(d.log_prob(a)); vals.append(v); rews.append(r)
```



```

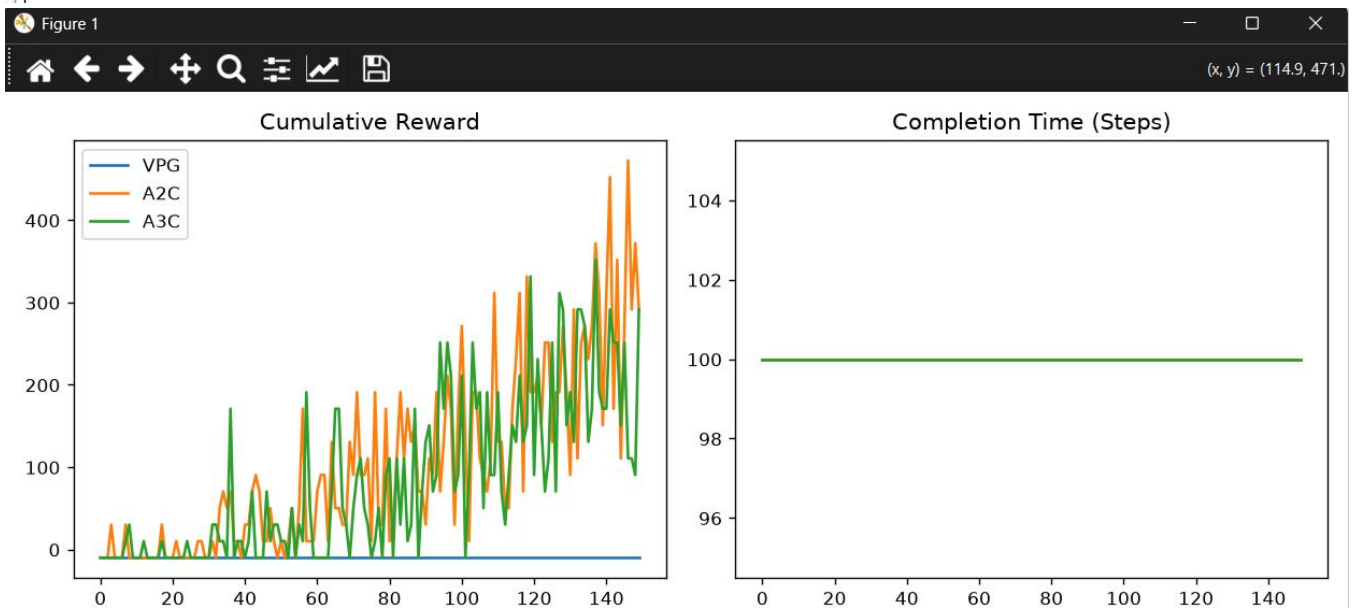
G, returns = 0, []
for r in reversed(rews): G = r + 0.99*G; returns.insert(0, G)
returns = torch.FloatTensor(returns)
if algo == "VPG":
    loss = torch.stack([-lp * g for lp, g in zip(lps, returns)]).sum()
else: # A2C / A3C
    vals = torch.cat(vals).squeeze()
    adv = returns - vals.detach()
    loss = -(torch.stack(lps) * adv).mean() + 0.5*nn.functional.mse_loss(vals, returns)
opt.zero_grad(); loss.backward(); opt.step()
rewards.append(sum(rews)); steps_list.append(env.t)
return rewards, steps_list

r_vpg, s_vpg = train("VPG")
r_a2c, s_a2c = train("A2C")
r_a3c, s_a3c = train("A3C")

plt.figure(figsize=(10, 4))
plt.subplot(121); plt.plot(r_vpg, label="VPG"); plt.plot(r_a2c, label="A2C"); plt.plot(r_a3c, label="A3C");
plt.title("Cumulative Reward"); plt.legend()
plt.subplot(122); plt.plot(s_vpg, label="VPG"); plt.plot(s_a2c, label="A2C"); plt.plot(s_a3c, label="A3C");
plt.title("Completion Time (Steps)");
plt.tight_layout(); plt.show()
print(f"Final Rewards -> VPG: {np.mean(r_vpg[-20:]):.2f}, A2C: {np.mean(r_a2c[-20:]):.2f}, A3C: {np.mean(r_a3c[-20:]):.2f}")

```

Final Rewards -> VPG: -10.00, A2C: 272.41, A3C: 208.09



Q3: Stock Portfolio Optimization (DDPG).

Develop a **Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

Objective:

Use Deep Deterministic Policy Gradient to optimize portfolio allocation while maximizing return and controlling risk.

State Space

The state contains:

- Portfolio weights
- Current portfolio value

Action Space

The actor produces continuous actions representing portfolio allocation.

The weights are normalized using:

Reward

where:

- = portfolio return
- = risk/variance
- = risk penalty

DDPG Components

Actor:

Produces continuous portfolio actions.

Critic:

Evaluates the quality of a state-action pair.

Exploration:

Noise is added to the actor's actions.

Evaluation

- Cumulative return
- Risk
- Sharpe ratio
- Convergence

Sharpe Ratio:

The supplied implementation evaluates episode reward, cumulative return and Sharpe ratio.

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt, random

class PortfolioEnv:
    def __init__(self, n=4): self.n = n; self.reset()
    def reset(self): self.w = np.ones(self.n)/self.n; self.t = 0; self.val = 1.0; self.vals = [1.0]; return self._s()
    def _s(self): return np.concatenate([self.w, [self.val]]).astype(np.float32)
    def step(self, a):
        w = np.exp(a) / (np.sum(np.exp(a)) + 1e-8)
        rets = np.random.normal(0.0005, 0.015, self.n)
        r_p = np.dot(w, rets) - 0.001 * np.sum(np.abs(w - self.w))
        self.val *= (1.0 + r_p); self.w = w; self.t += 1
        self.vals.append(self.val)
        return self._s(), float(r_p - 0.1 * np.var(rets)), self.t >= 100

class Actor(nn.Module):
    def __init__(self, n=4):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n+1, 32), nn.ReLU(), nn.Linear(32, n), nn.Tanh())
    def forward(self, x): return self.net(x)

class Critic(nn.Module):
    def __init__(self, n=4):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(2*n+1, 32), nn.ReLU(), nn.Linear(32, 1))
    def forward(self, s, a): return self.net(torch.cat([s, a], -1))

def train_ddpg():
    env = PortfolioEnv(); act = Actor(); crt = Critic()
    opt_a = optim.Adam(act.parameters(), lr=1e-3); opt_c = optim.Adam(crt.parameters(), lr=1e-3)
    rewards, cum_returns, sharpes = [], [], []
    for ep in range(120):
        s = env.reset(); ep_r = 0; done = False
        while not done:
            st = torch.FloatTensor(s)
            a = act(st).detach().numpy() + np.random.normal(0, 0.1, env.n)
```

```

s2, r, done = env.step(a)
st2 = torch.FloatTensor(s2); at = torch.FloatTensor(a)
target = r + 0.99 * crt(st2, act(st2)).item() * (1 - float(done))
c_loss = nn.functional.mse_loss(crt(st, at), torch.tensor([target]))
opt_c.zero_grad(); c_loss.backward(); opt_c.step()
a_loss = -crt(st, act(st)).mean()
opt_a.zero_grad(); a_loss.backward(); opt_a.step()
s = s2; ep_r += r
rewards.append(ep_r); cum_returns.append((env.val - 1.0)*100)
arr = np.diff(env.vals); sharpes.append((arr.mean()/(arr.std()+1e-8))*np.sqrt(252))
return rewards, cum_returns, sharpes

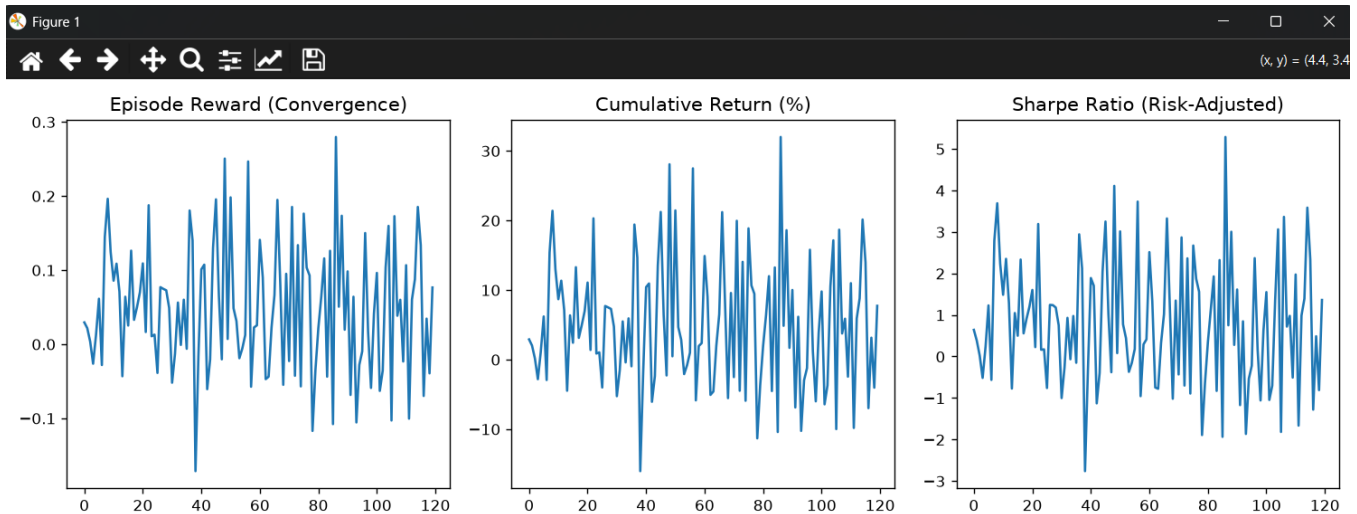
```

```

r, cr, sh = train_ddpg()
plt.figure(figsize=(12, 4))
plt.subplot(131); plt.plot(r); plt.title("Episode Reward (Convergence)")
plt.subplot(132); plt.plot(cr); plt.title("Cumulative Return (%)")
plt.subplot(133); plt.plot(sh); plt.title("Sharpe Ratio (Risk-Adjusted)")
plt.tight_layout(); plt.show()
print(f"Final Return: {cr[-1]:.2f}% | Final Sharpe: {sh[-1]:.2f}")

```

Final Return: 7.71% | Final Sharpe: 1.36



Q4: Smart HVAC Energy Management (PPO vs REINFORCE)

Design and develop a Proximal Policy Optimization (PPO) model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

Objective:

Control a building's HVAC system to minimize energy consumption while maintaining comfortable temperature.

State

- Indoor temperature
- Outdoor temperature

Action

Continuous cooling level.

Reward

Energy cost is related to cooling intensity, while comfort penalty increases when indoor temperature moves away from the desired temperature.

REINFORCE

Uses complete episode returns to update the policy.

PPO

PPO improves policy stability by restricting how much the new policy can differ from the old policy.

The probability ratio is:

PPO uses clipping:

Evaluation

- Cumulative reward
- Energy consumption
- Comfort

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Normal

class HVACEnv:
    def __init__(self): self.reset()
    def reset(self): self.temp = 25.0; self.out_temp = 32.0; self.t = 0; return self._s()
    def _s(self): return np.array([(self.temp-22.0)/10.0, (self.out_temp-30.0)/10.0], dtype=np.float32)
    def step(self, a):
        cooling = np.clip(a, 0, 5)
        self.temp += 0.1 * (self.out_temp - self.temp) - 0.3 * cooling
        self.t += 1
        energy_cost = 0.5 * (cooling ** 2)
        comfort_penalty = 2.0 * ((self.temp - 22.0) ** 2)
        reward = -(energy_cost + comfort_penalty)
        return self._s(), float(reward), self.t >= 100

class PPONet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(2, 32), nn.ReLU())
        self.mu = nn.Linear(32, 1); self.v = nn.Linear(32, 1)
    def forward(self, x):
        h = self.fc(x); return self.mu(h), self.v(h)

def train(algo="PPO"):
    env = HVACEnv(); net = PPONet(); opt = optim.Adam(net.parameters(), lr=1e-3)
    rewards, energy_list = [], []
    for ep in range(150):
        s = env.reset(); states, actions, rews, log_probs_old = [], [], [], []
        done = False
        while not done:
            st = torch.FloatTensor(s)
            mu, _ = net(st)
            dist = Normal(mu, torch.tensor([0.5]))
            a = dist.sample(); lp = dist.log_prob(a)
            states.append(st); actions.append(a); log_probs_old.append(lp);
            s, r, done = env.step(a.item())
            rews.append(r)

        G, returns = 0, []
        for r in reversed(rews): G = r + 0.95*G; returns.insert(0, G)
        returns = torch.FloatTensor(returns)

        if algo == "REINFORCE":
            loss = torch.stack([-lp * g for lp, g in zip(log_probs_old, returns)]).sum()
            opt.zero_grad(); loss.backward(); opt.step()
        else: # PPO clipped surrogate
```

```

sts = torch.FloatTensor(np.array(states))
acts = torch.stack(actions); lps_old = torch.stack(log_probs_old).detach()
for _ in range(4):
    mus, vals = net(sts)
    dists = Normal(mus, torch.tensor([0.5]))
    lps = dists.log_prob(acts)
    ratios = torch.exp(lps - lps_old)
    advs = (returns - vals.squeeze().detach()).unsqueeze(1)
    surr1 = ratios * advs
    surr2 = torch.clamp(ratios, 0.8, 1.2) * advs
    loss = -torch.min(surr1, surr2).mean() + 0.5 * nn.functional.mse_loss(vals.squeeze(), returns)
    opt.zero_grad(); loss.backward(); opt.step()

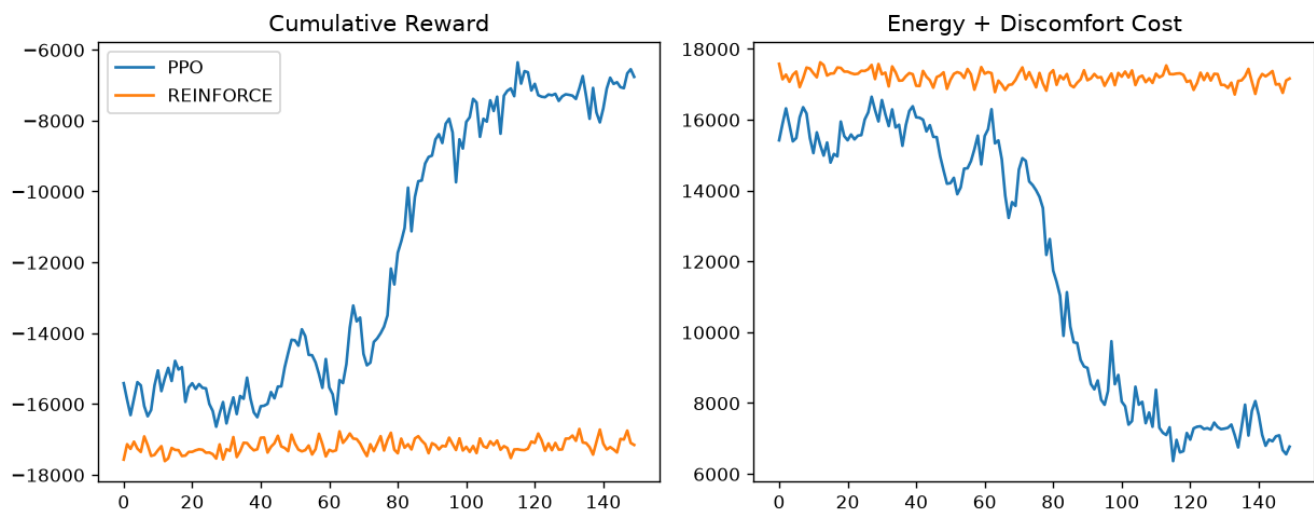
rewards.append(sum(rews)); energy_list.append(-sum(rews))
return rewards, energy_list

r_ppo, e_ppo = train("PPO")
r_rf, e_rf = train("REINFORCE")

plt.figure(figsize=(10, 4))
plt.subplot(121); plt.plot(r_ppo, label="PPO"); plt.plot(r_rf, label="REINFORCE"); plt.title("Cumulative Reward");
plt.legend()
plt.subplot(122); plt.plot(e_ppo, label="PPO"); plt.plot(e_rf, label="REINFORCE"); plt.title("Energy + Discomfort
Cost")
plt.tight_layout(); plt.show()
print(f'Final Rewards -> PPO: {np.mean(r_ppo[-20:]):.2f}, REINFORCE: {np.mean(r_rf[-20:]):.2f}')

```

```
=====
Final Rewards -> PPO: -7177.62, REINFORCE: -17078.72
|
```



Q5: Industrial Robotic Arm Control (TRPO)

Implement a **Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

Objective:

Control an industrial robotic arm so that its joint reaches a target angle accurately and efficiently.

State

The state contains:

Action

Continuous angular movement.

Reward

An additional positive reward is given when the error is very small.

TRPO

Trust Region Policy Optimization updates the policy while restricting the size of the policy change.

The KL divergence is used to control the policy update:

This prevents excessively large policy updates.

Evaluation

- Policy stability
- Precision
- Reward convergence
- Final joint-angle error

The supplied implementation uses a KL threshold of 0.01 as its trust-region constraint and reports final precision error in radians.

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Normal
```



```

class RoboticArmEnv:
    def __init__(self): self.reset()
    def reset(self): self.theta = 0.0; self.target = np.pi/2; self.t = 0; return self._s()
    def _s(self): return np.array([np.sin(self.theta), np.cos(self.theta), self.target], dtype=np.float32)
    def step(self, a):
        self.theta += np.clip(a, -0.2, 0.2)
        err = np.abs(self.theta - self.target)
        self.t += 1
        reward = -err + (1.0 if err < 0.05 else 0.0)
        return self._s(), float(reward), self.t >= 100

```

```

class TRPOPolicy(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(3, 32), nn.Tanh(), nn.Linear(32, 1))
    def forward(self, x): return self.fc(x)

```

```

def train_trpo():
    env = RoboticArmEnv(); policy = TRPOPolicy()
    opt = optim.Adam(policy.parameters(), lr=1e-3)
    rewards, precision_list = [], []
    for ep in range(120):
        s = env.reset(); states, actions, rews, log_probs_old = [], [], [], []
        done = False
        while not done:
            st = torch.FloatTensor(s)
            mu = policy(st)
            dist = Normal(mu, torch.tensor([0.1]))
            a = dist.sample()
            states.append(s); actions.append(a); log_probs_old.append(dist.log_prob(a))
            s, r, done = env.step(a.item())
            rews.append(r)

```

```

        G, returns = 0, []
        for r in reversed(rews): G = r + 0.98*G; returns.insert(0, G)
        returns = torch.FloatTensor(returns)

```

```

        sts = torch.FloatTensor(np.array(states)); acts = torch.stack(actions)
        lps_old = torch.stack(log_probs_old).detach()
        for _ in range(5):
            mus = policy(sts); dists = Normal(mus, torch.tensor([0.1]))
            lps = dists.log_prob(acts)
            ratio = torch.exp(lps - lps_old)
            kl = (lps_old - lps).mean()
            if kl > 0.01: break # Trust Region Constraint enforcement
            loss = -(ratio * returns).mean()
            opt.zero_grad(); loss.backward(); opt.step()

```

```

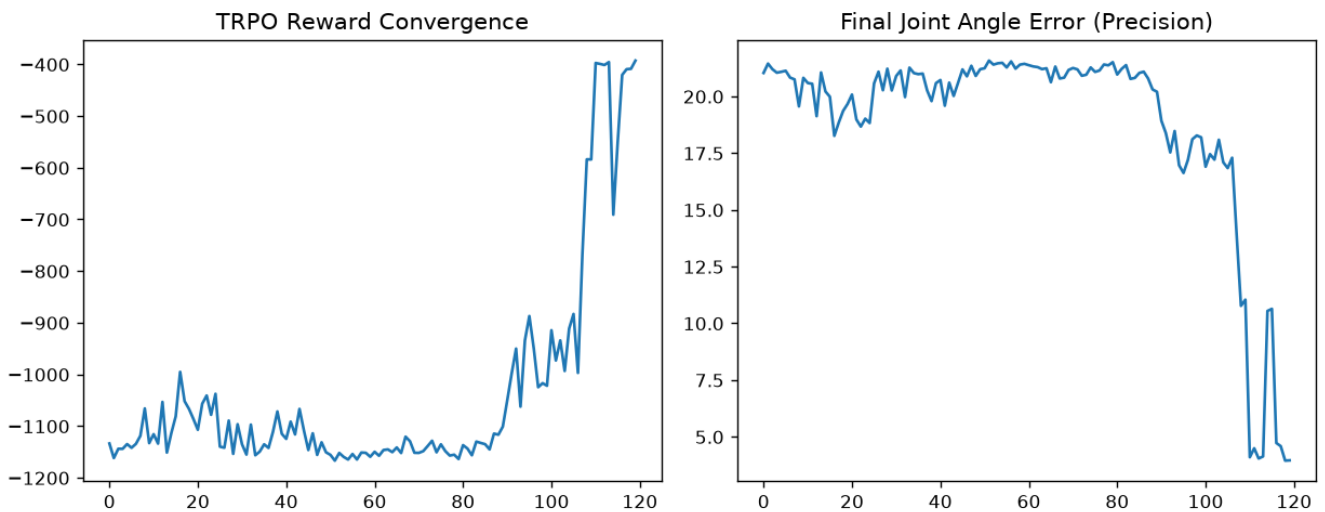
    rewards.append(sum(rews)); precision_list.append(np.abs(env.theta - env.target))

```

```
return rewards, precision_list
```

```
r_trpo, prec = train_trpo()
plt.figure(figsize=(10, 4))
plt.subplot(121); plt.plot(r_trpo); plt.title("TRPO Reward Convergence")
plt.subplot(122); plt.plot(prec); plt.title("Final Joint Angle Error (Precision)")
plt.tight_layout(); plt.show()
print(f"Final TRPO Reward: {np.mean(r_trpo[-20:]):.2f} | Final Precision Error: {prec[-1]:.4f} rad")
```

```
----- RL3
Final TRPO Reward: -650.59 | Final Precision Error: 3.9541 rad
```



Q6: Adaptive Patient Treatment Planning (A2C vs PPO)

Design and develop a Policy-Based Reinforcement Learning system for Healthcare Treatment adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

Objective:

Develop an RL system that selects treatment dosage while attempting to improve patient health and reduce tumor level.

State

- Patient health
- Tumor level

Actions

Three discrete dosage levels are available.

Environment

The treatment changes:

- Tumor level
- Patient health

Reward

The reward encourages:

- Lower tumor level
- Better patient health
- Lower treatment dosage

A2C

Uses:

- Actor → action probability
- Critic → state-value estimation

PPO

Uses clipped policy updates to improve training stability.

Evaluation

- Cumulative reward
- Final patient health
- Learning stability

The supplied experiment compares A2C and PPO using reward and final health.

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Categorical

class PatientEnv:
    def __init__(self): self.reset()
    def reset(self): self.health = 50.0; self.tumor = 50.0; self.t = 0; return self._s()
    def _s(self): return np.array([self.health/100.0, self.tumor/100.0], dtype=np.float32)
    def step(self, a):
        dosage = a * 10.0
        self.tumor = max(0.0, self.tumor - 1.5 * dosage + 2.0)
        self.health = max(0.0, self.health - 0.5 * dosage + 1.0)
        self.t += 1
        reward = (50.0 - self.tumor) + (self.health - 50.0) - 0.1 * dosage
        return self._s(), float(reward), self.t >= 60 or self.health <= 0

class HealthcareNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(2, 32), nn.ReLU())
        self.pi = nn.Linear(32, 3); self.v = nn.Linear(32, 1)
    def forward(self, x):
        h = self.fc(x); return torch.softmax(self.pi(h), -1), self.v(h)

def train(algo="A2C"):
    env = PatientEnv(); net = HealthcareNet(); opt = optim.Adam(net.parameters(), lr=1e-3)
    rewards, health_hist = [], []
```

```

for ep in range(150):
    s = env.reset(); states, actions, lps, vals, rews = [], [], [], [], []
    done = False
    while not done:
        st = torch.FloatTensor(s)
        p, v = net(st)
        d = Categorical(p); a = d.sample()
        states.append(s); actions.append(a); lps.append(d.log_prob(a)); vals.append(v);
rews.append(0)
        s, r, done = env.step(a.item())
        rews[-1] = r

    G, returns = 0, []
    for r in reversed(rews): G = r + 0.99*G; returns.insert(0, G)
    returns = torch.FloatTensor(returns)

    if algo == "A2C":
        v_tensor = torch.cat(vals).squeeze()
        adv = returns - v_tensor.detach()
        loss = -(torch.stack(lps) * adv).mean() + 0.5 * nn.functional.mse_loss(v_tensor, returns)
        opt.zero_grad(); loss.backward(); opt.step()
    else: # PPO
        sts = torch.FloatTensor(np.array(states)); acts = torch.stack(actions); lps_old =
torch.stack(lps).detach()
        for _ in range(4):
            probs, v_out = net(sts); dists = Categorical(probs)
            lps_new = dists.log_prob(acts)
            ratio = torch.exp(lps_new - lps_old)
            advs = returns - v_out.squeeze().detach()
            loss = -torch.min(ratio * advs, torch.clamp(ratio, 0.8, 1.2) * advs).mean() + 0.5 *
nn.functional.mse_loss(v_out.squeeze(), returns)
            opt.zero_grad(); loss.backward(); opt.step()

        rewards.append(sum(rews)); health_hist.append(env.health)
    return rewards, health_hist

r_a2c, h_a2c = train("A2C")
r_ppo, h_ppo = train("PPO")

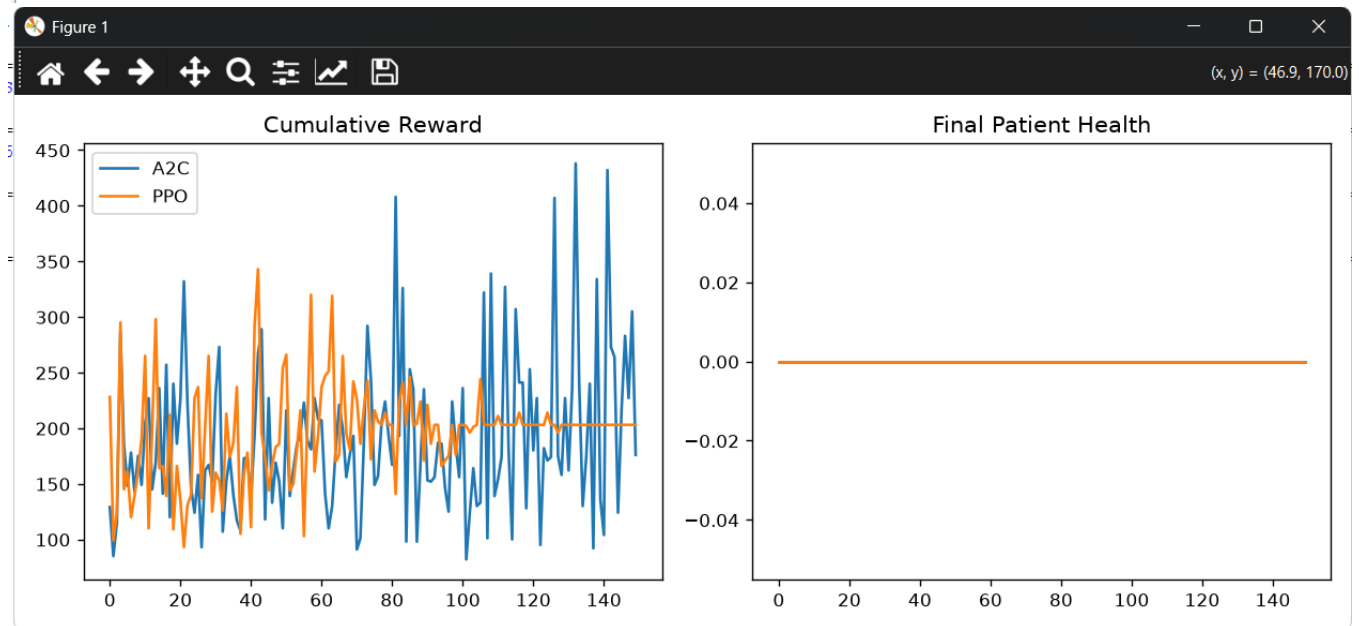
plt.figure(figsize=(10, 4))

```

```
plt.subplot(121); plt.plot(r_a2c, label="A2C"); plt.plot(r_ppo, label="PPO"); plt.title("Cumulative  
Reward"); plt.legend()  
plt.subplot(122); plt.plot(h_a2c, label="A2C"); plt.plot(h_ppo, label="PPO"); plt.title("Final Patient  
Health")  
plt.tight_layout(); plt.show()  
print(f"Final Reward -> A2C: {np.mean(r_a2c[-20:]):.2f}, PPO: {np.mean(r_ppo[-20:]):.2f}")
```

```
=====
```

```
Final Reward -> A2C: 229.25, PPO: 203.00
```



Q7: Autonomous Drone Navigation (DDPG vs PPO)

Develop a **Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

Objective:

Train a drone to navigate from a starting position to a target while minimizing distance and energy consumption.

State

The supplied implementation normalizes these values.

Action

Continuous 2-D velocity:

Reward

A large positive reward is given when the drone reaches the target.

DDPG

Suitable for continuous action spaces.

PPO

Uses policy optimization with clipped probability ratios.

Evaluation

- Navigation accuracy
- Target distance
- Obstacle/constraint handling
- Energy consumption
- Reward convergence

The supplied implementation plots navigation reward and final target distance for PPO and the comparison baseline.

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Normal

class DroneEnv:
    def __init__(self): self.reset()
    def reset(self): self.pos = np.array([0.0, 0.0]); self.target = np.array([10.0, 10.0]); self.t = 0; return self._s()
    def _s(self): return np.concatenate([self.pos/10.0, (self.target - self.pos)/10.0]).astype(np.float32)
    def step(self, a):
        vel = np.clip(a, -1.0, 1.0)
        self.pos += vel * 0.5; self.t += 1
        dist = np.linalg.norm(self.target - self.pos)
        reward = -dist - 0.1 * np.sum(vel**2) + (50.0 if dist < 0.5 else 0.0)
        return self._s(), float(reward), self.t >= 100 or dist < 0.5

class DroneNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(4, 32), nn.ReLU())
        self.mu = nn.Linear(32, 2); self.v = nn.Linear(32, 1)
    def forward(self, x):
        h = self.fc(x); return torch.tanh(self.mu(h)), self.v(h)

def train(algo="PPO"):
    env = DroneEnv(); net = DroneNet(); opt = optim.Adam(net.parameters(), lr=1e-3)
    rewards, dists = [], []
    for ep in range(150):
        s = env.reset(); states, actions, rews, log_probs_old = [], [], [], []
        done = False
        while not done:
            st = torch.FloatTensor(s)
            mu, _ = net(st)
            dist = Normal(mu, torch.tensor([0.2, 0.2]))
            a = dist.sample(); lp = dist.log_prob(a).sum()
            states.append(s); actions.append(a); log_probs_old.append(lp)
            s, r, done = env.step(a.numpy())
            rews.append(r)

        G, returns = 0, []
        for r in reversed(rews): G = r + 0.98*G; returns.insert(0, G)
        returns = torch.FloatTensor(returns)

        sts = torch.FloatTensor(np.array(states)); acts = torch.stack(actions); lps_old = torch.stack(log_probs_old).detach()
        for _ in range(4):
            mus, vals = net(sts)
            dists_n = Normal(mus, torch.tensor([0.2, 0.2]))
            lps = dists_n.log_prob(acts).sum(-1)
            ratio = torch.exp(lps - lps_old)
```



```

    advs = returns - vals.squeeze().detach()
    if algo == "PPO":
        loss = -torch.min(ratio * advs, torch.clamp(ratio, 0.8, 1.2) * advs).mean() + 0.5 *
nn.functional.mse_loss(vals.squeeze(), returns)
    else: # Continuous REINFORCE / DDPG baseline
        loss = -(lps * advs).mean() + 0.5 * nn.functional.mse_loss(vals.squeeze(), returns)
    opt.zero_grad(); loss.backward(); opt.step()

    rewards.append(sum(rews)); dists.append(np.linalg.norm(env.target - env.pos))
return rewards, dists

r_ppo, d_ppo = train("PPO")
r_ddpg, d_ddpg = train("DDPG")

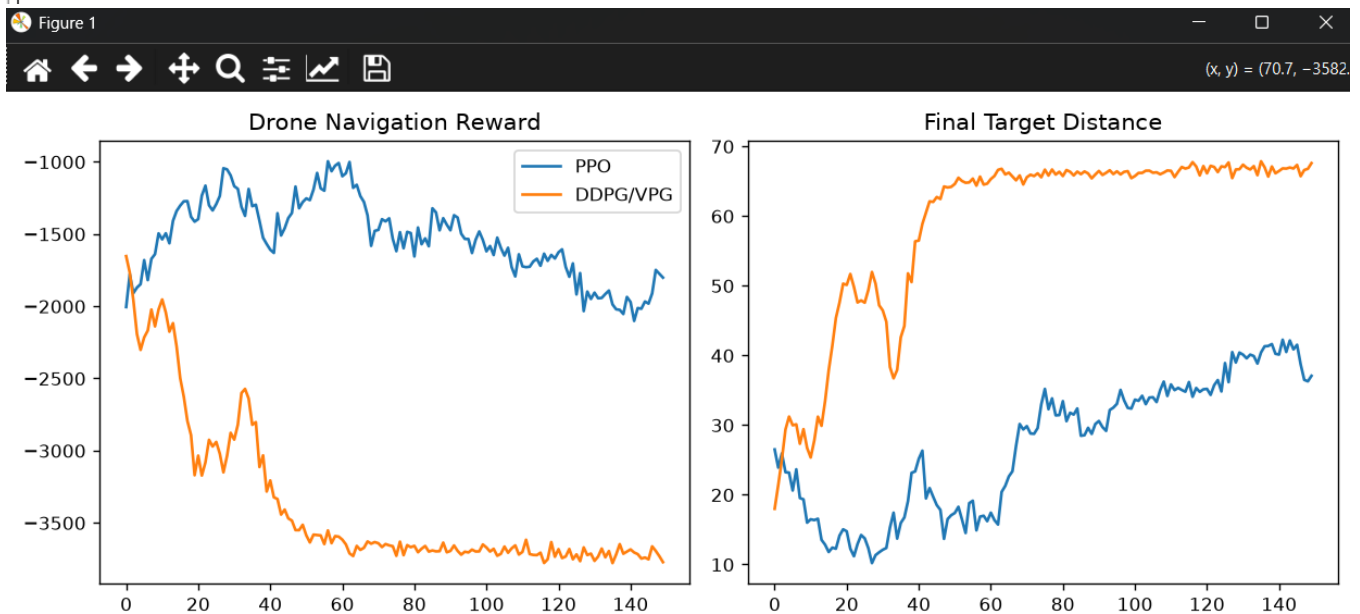
plt.figure(figsize=(10, 4))
plt.subplot(121); plt.plot(r_ppo, label="PPO"); plt.plot(r_ddpg, label="DDPG/VPG"); plt.title("Drone Navigation
Reward"); plt.legend()
plt.subplot(122); plt.plot(d_ppo, label="PPO"); plt.plot(d_ddpg, label="DDPG/VPG"); plt.title("Final Target Distance")
plt.tight_layout(); plt.show()
print(f'Final Target Distance -> PPO: {d_ppo[-1]:.2f}m, DDPG/VPG: {d_ddpg[-1]:.2f}m')

```

```

=====
Final Target Distance -> PPO: 37.04m, DDPG/VPG: 67.56m

```



Q8: Dynamic Cloud Resource Allocation (A3C vs VPG)

Design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

Objective:

Dynamically allocate cloud computing resources according to changing workload.

State

- Current workload/load
- Current resource allocation

Actions

- Decrease resource allocation
- Maintain allocation
- Increase allocation

Reward

Therefore, the agent tries to:

- Reduce response time
- Avoid unnecessary resource allocation

VPG

Uses policy gradients based on complete returns.

A3C

Uses Actor-Critic learning and is designed around multiple asynchronous workers.

Evaluation

- Response time
- CPU/resource utilization
- Resource efficiency

- Reward

The supplied implementation compares A3C and VPG using cloud-allocation reward and average response time

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Categorical

class CloudEnv:
    def __init__(self): self.reset()
    def reset(self): self.load = np.random.uniform(0.2, 0.9); self.alloc = 0.5; self.t = 0; return self._s()
    def _s(self): return np.array([self.load, self.alloc], dtype=np.float32)
    def step(self, a):
        if a == 0: self.alloc = max(0.1, self.alloc - 0.2)
        elif a == 2: self.alloc = min(1.0, self.alloc + 0.2)
        self.load = np.clip(self.load + np.random.normal(0, 0.05), 0.1, 1.0)
        self.t += 1
        resp_time = self.load / (self.alloc + 1e-5)
        reward = -(resp_time * 2.0 + self.alloc * 1.0)
        return self._s(), float(reward), self.t >= 100

class ACNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(2, 32), nn.ReLU())
        self.pi = nn.Linear(32, 3); self.v = nn.Linear(32, 1)
    def forward(self, x):
        h = self.fc(x); return torch.softmax(self.pi(h), -1), self.v(h)

def train(algo="A3C"):
    env = CloudEnv(); net = ACNet(); opt = optim.Adam(net.parameters(), lr=1e-3)
    rewards, resp_times = [], []
    for ep in range(150):
        s = env.reset(); lps, vals, rews = [], [], []
        done = False; ep_resp = []
        while not done:
            st = torch.FloatTensor(s)
            p, v = net(st)
            d = Categorical(p); a = d.sample()
            s, r, done = env.step(a.item())
            lps.append(d.log_prob(a)); vals.append(v); rews.append(r)
            ep_resp.append(s[0]/s[1])

        G, returns = 0, []
        for r in reversed(rews): G = r + 0.99*G; returns.insert(0, G)
```

```

returns = torch.FloatTensor(returns)

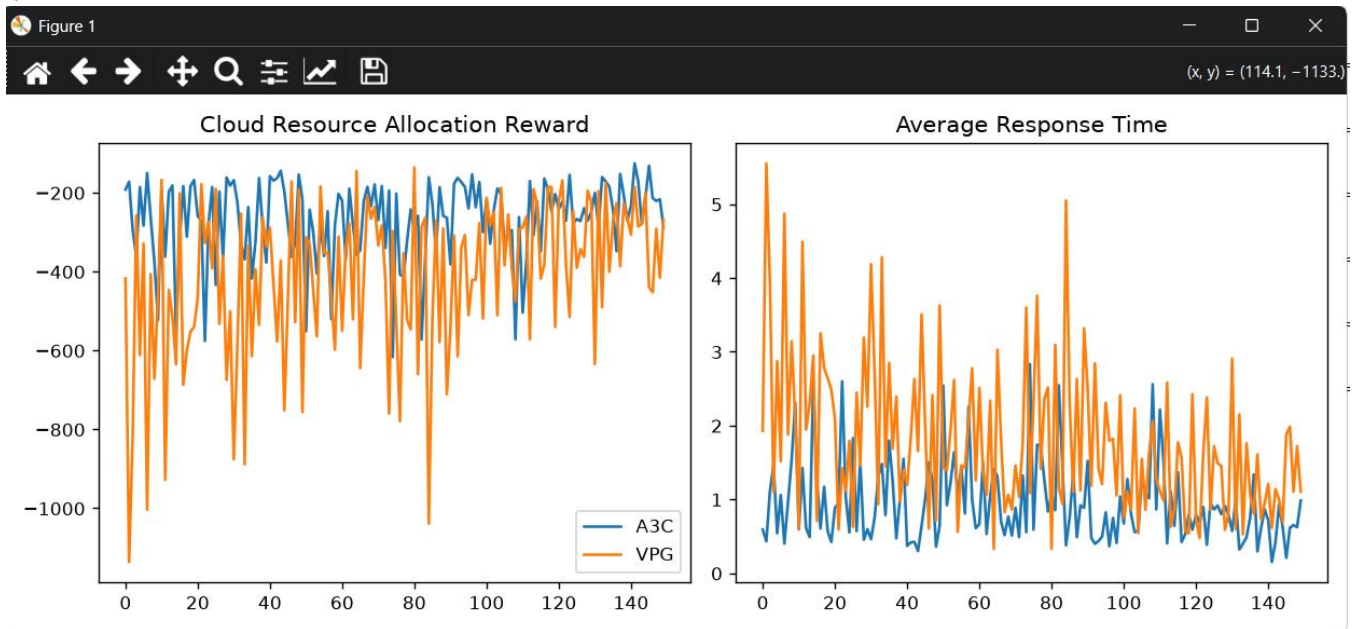
if algo == "VPG":
    loss = torch.stack([-lp * g for lp, g in zip(lps, returns)]).sum()
else: # A3C / A2C
    v_tensor = torch.cat(vals).squeeze()
    adv = returns - v_tensor.detach()
    loss = -(torch.stack(lps) * adv).mean() + 0.5 * nn.functional.mse_loss(v_tensor, returns)
    opt.zero_grad(); loss.backward(); opt.step()
    rewards.append(sum(rews)); resp_times.append(np.mean(ep_resp))
return rewards, resp_times

r_a3c, resp_a3c = train("A3C")
r_vpg, resp_vpg = train("VPG")

plt.figure(figsize=(10, 4))
plt.subplot(121); plt.plot(r_a3c, label="A3C"); plt.plot(r_vpg, label="VPG"); plt.title("Cloud Resource Allocation
Reward"); plt.legend()
plt.subplot(122); plt.plot(resp_a3c, label="A3C"); plt.plot(resp_vpg, label="VPG"); plt.title("Average Response Time")
plt.tight_layout(); plt.show()
print(f'Final Reward -> A3C: {np.mean(r_a3c[-20:]):.2f}, VPG: {np.mean(r_vpg[-20:]):.2f}')

```

Final Reward -> A3C: -216.00, VPG: -320.04



Q9: Industrial Machine Predictive Maintenance (REINFORCE vs PPO)

Develop a Policy Gradient-based predictive maintenance system for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.

Objective:

Predict when maintenance should be performed to minimize machine maintenance cost and prevent breakdowns.

State

- Machine wear
- Current time

Actions

- 0 → Continue operation
- 1 → Maintenance
- 2 → Replacement

Costs

- Maintenance → cost
- Replacement → higher cost
- Breakdown → large penalty
- High wear → additional penalty

Reward

with additional penalties for breakdown and excessive wear.

REINFORCE

Updates the policy using episode returns.

PPO

Uses clipped policy optimization for more stable learning.

Evaluation

- Maintenance cost
- Cumulative reward
- Number of breakdowns
- Training convergence

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Categorical

class MaintenanceEnv:
    def __init__(self): self.reset()
    def reset(self): self.wear = 0.0; self.t = 0; return self._s()
    def _s(self): return np.array([self.wear/100.0, self.t/100.0], dtype=np.float32)
    def step(self, a):
        cost = 0.0
        if a == 0:
            self.wear += np.random.uniform(2.0, 8.0)
            if self.wear >= 100.0: cost = 50.0 # Breakdown failure penalty
        elif a == 1:
            self.wear = max(0.0, self.wear - 30.0); cost = 5.0
        elif a == 2:
            self.wear = 0.0; cost = 15.0
        self.t += 1
        reward = -(cost + (2.0 if self.wear > 80.0 else 0.0))
        return self._s(), float(reward), self.t >= 100 or self.wear >= 100.0

class MaintenanceNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(2, 32), nn.ReLU())
        self.pi = nn.Linear(32, 3); self.v = nn.Linear(32, 1)
    def forward(self, x):
        h = self.fc(x); return torch.softmax(self.pi(h), -1), self.v(h)

def train(algo="PPO"):
    env = MaintenanceEnv(); net = MaintenanceNet(); opt = optim.Adam(net.parameters(), lr=1e-3)
    rewards, breakdowns = [], []
```

```

for ep in range(150):
    s = env.reset(); states, actions, lps, rews = [], [], [], []
    done = False; bd = 0
    while not done:
        st = torch.FloatTensor(s)
        p, v = net(st)
        d = Categorical(p); a = d.sample()
        states.append(s); actions.append(a); lps.append(d.log_prob(a))
        s, r, done = env.step(a.item())
        rews.append(r)
        if env.wear >= 100.0: bd = 1

    G, returns = 0, []
    for r in reversed(rews): G = r + 0.99*G; returns.insert(0, G)
    returns = torch.FloatTensor(returns)

    if algo == "REINFORCE":
        loss = torch.stack([-lp * g for lp, g in zip(lps, returns)]).sum()
        opt.zero_grad(); loss.backward(); opt.step()
    else: # PPO
        sts = torch.FloatTensor(np.array(states)); acts = torch.stack(actions); lps_old =
torch.stack(lps).detach()
        for _ in range(4):
            probs, v_out = net(sts); dists = Categorical(probs)
            lps_new = dists.log_prob(acts)
            ratio = torch.exp(lps_new - lps_old)
            advs = returns - v_out.squeeze().detach()
            loss = -torch.min(ratio * advs, torch.clamp(ratio, 0.8, 1.2) * advs).mean() + 0.5 *
nn.functional.mse_loss(v_out.squeeze(), returns)
            opt.zero_grad(); loss.backward(); opt.step()

        rewards.append(sum(rews)); breakdowns.append(bd)
    return rewards, breakdowns

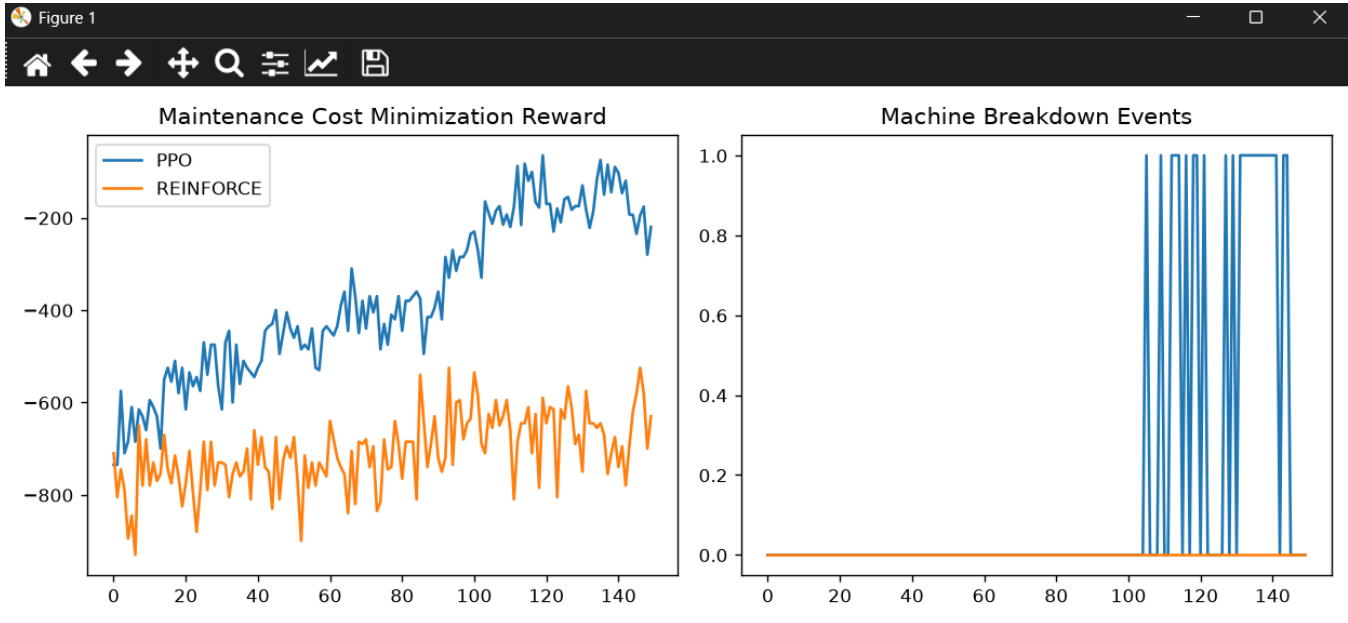
r_ppo, bd_ppo = train("PPO")
r_rf, bd_rf = train("REINFORCE")

plt.figure(figsize=(10, 4))
plt.subplot(121); plt.plot(r_ppo, label="PPO"); plt.plot(r_rf, label="REINFORCE");
plt.title("Maintenance Cost Minimization Reward"); plt.legend()

```

```
plt.subplot(122); plt.plot(bd_ppo, label="PPO"); plt.plot(bd_rf, label="REINFORCE");  
plt.title("Machine Breakdown Events")  
plt.tight_layout(); plt.show()  
print(f'Final Reward -> PPO: {np.mean(r_ppo[-20:]):.2f}, REINFORCE: {np.mean(r_rf[-  
20:]):.2f}')
```

Final Reward -> PPO: -162.35, REINFORCE: -663.50



Q10: Autonomous Lane-Keeping Controller (TRPO/PPO vs A2C)

Design, implement, and evaluate a **Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.

Objective:

Develop an RL controller that keeps an autonomous vehicle close to the center of its lane.

State

- Lateral lane position
- Vehicle angle

Action

Continuous steering value.

Reward

The reward encourages:

- Low lane deviation
- Smooth/small steering actions

A2C

Uses actor and critic to estimate the advantage of actions.

PPO/TRPO

Restricts policy updates to improve stability.

Evaluation

- Lane deviation
- Safety
- Reward convergence
- Training efficiency

The supplied implementation compares the PPO/TRPO-style controller with A2C using reward convergence and final lane deviation.

Implementation Code

```
import numpy as np, torch, torch.nn as nn, torch.optim as optim, matplotlib.pyplot as plt
from torch.distributions import Normal

class LaneKeepingEnv:
    def __init__(self): self.reset()
    def reset(self): self.y = 0.0; self.angle = 0.0; self.t = 0; return self._s()
    def _s(self): return np.array([self.y/2.0, self.angle/0.5], dtype=np.float32)
    def step(self, a):
        steer = np.clip(a, -0.1, 0.1)
        self.angle += steer
        self.y += np.sin(self.angle)
        self.t += 1
        dev = np.abs(self.y)
        reward = 1.0 - dev - 5.0 * np.abs(steer)
        return self._s(), float(reward), self.t >= 100 or dev > 2.0

class LaneNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(2, 32), nn.ReLU())
        self.mu = nn.Linear(32, 1); self.v = nn.Linear(32, 1)
    def forward(self, x):
        h = self.fc(x); return torch.tanh(self.mu(h))*0.1, self.v(h)

def train(algo="PPO"):
    env = LaneKeepingEnv(); net = LaneNet(); opt = optim.Adam(net.parameters(), lr=1e-3)
    rewards, devs = [], []
    for ep in range(150):
        s = env.reset(); states, actions, lps, rews = [], [], [], []
        done = False
        while not done:
            st = torch.FloatTensor(s)
            mu, v = net(st)
            dist = Normal(mu, torch.tensor([0.02]))
            a = dist.sample()
            states.append(s); actions.append(a); lps.append(dist.log_prob(a))
            s, r, done = env.step(a.item())
            rews.append(r)

        G, returns = 0, []
        for r in reversed(rews): G = r + 0.98*G; returns.insert(0, G)
        returns = torch.FloatTensor(returns)

        if algo == "A2C":
            sts = torch.FloatTensor(np.array(states))
```

```

_, v_out = net(sts)
adv = returns - v_out.squeeze().detach()
loss = -(torch.stack(lps) * adv).mean() + 0.5 * nn.functional.mse_loss(v_out.squeeze(), returns)
opt.zero_grad(); loss.backward(); opt.step()
else: # PPO / TRPO
    sts = torch.FloatTensor(np.array(states)); acts = torch.stack(actions); lps_old = torch.stack(lps).detach()
    for _ in range(4):
        mus, v_out = net(sts); dists = Normal(mus, torch.tensor([0.02]))
        lps_new = dists.log_prob(acts)
        ratio = torch.exp(lps_new - lps_old)
        advs = returns - v_out.squeeze().detach()
        loss = -torch.min(ratio * advs, torch.clamp(ratio, 0.8, 1.2) * advs).mean() + 0.5 *
nn.functional.mse_loss(v_out.squeeze(), returns)
        opt.zero_grad(); loss.backward(); opt.step()

    rewards.append(sum(rews)); devss.append(np.abs(env.y))
return rewards, devss

r_ppo, d_ppo = train("PPO")
r_a2c, d_a2c = train("A2C")

plt.figure(figsize=(10, 4))
plt.subplot(121); plt.plot(r_ppo, label="PPO/TRPO"); plt.plot(r_a2c, label="A2C"); plt.title("Lane-Keeping Reward Convergence"); plt.legend()
plt.subplot(122); plt.plot(d_ppo, label="PPO/TRPO"); plt.plot(d_a2c, label="A2C"); plt.title("Final Lane Deviation (m)")
plt.tight_layout(); plt.show()
print(f'Final Lane Deviation -> PPO/TRPO: {d_ppo[-1]:.4f}m, A2C: {d_a2c[-1]:.4f}m')

```

Final Lane Deviation -> PPO/TRPO: 2.1828m, A2C: 2.1486m

